

RAPPORT DE PROJET

Bases de Données Réparties

Scénario 1 ET 2

Nom / Prénom : BARHOINE Ayoub / MOUSSA Yassine

Classe / Groupe : 2ème FI GI

Introduction

Ce rapport détaille la conception, le déploiement et l'optimisation d'une base de données répartie à fragmentation horizontale, simulée à l'aide de conteneurs isolés Oracle Database Express Edition (XE). L'objectif de ce premier scénario est de valider les mécanismes de transparence de localisation, d'assurer la cohérence globale via des blocs PL/SQL (procédures et triggers), et d'analyser le comportement du moteur d'optimisation Oracle face à des requêtes distribuées fédérées.

La table LigneCommandes du projet EShop est fragmentée horizontalement entre deux sites distants selon un critère de quantité (seuil : 100) : les grosses commandes (Quantité $\geq$ 100) sont gérées par le Site 1, les petites (Quantité $<$ 100) par le Site 2.

1. Configuration Docker Compose

L'infrastructure repose sur trois conteneurs Oracle XE orchestrés via Docker Compose et interconnectés sur un réseau bridge isolé nommé oraclenet. La commande docker compose up -d lance simultanément les trois instances, puis docker ps valide leur bon démarrage.

```
PS C:\Users\Ideapad\Downloads\projetsql\TP_BDD_Distribuee> docker compose up -d
te 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 3/3
e9c071f5d172      host          host          local
8318794c7ec9      none         null          local
4390c7809781      oracle-plsql-project_default bridge        local
840148870ddf      tp_bdd_distribuee_oraclenet bridge        local
PS C:\Users\Ideapad\Downloads\projetsql\TP_BDD_Distribuee> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
fe6f2739c6d6   gvenzl/oracle-xe:latest "container-entrypoint..." 41 hours ago   Up 2 hours   0.0.0.0:1524->1521/tcp, [::]:1524->1521/tcp orac
le-main
e1701734b74c   gvenzl/oracle-xe:latest "container-entrypoint..." 41 hours ago   Up 2 hours   0.0.0.0:1523->1521/tcp, [::]:1523->1521/tcp orac
le-site2
1752e3b57d66   gvenzl/oracle-xe:latest "container-entrypoint..." 41 hours ago   Up 2 hours   0.0.0.0:1522->1521/tcp, [::]:1522->1521/tcp orac
le-site1
PS C:\Users\Ideapad\Downloads\projetsql\TP_BDD_Distribuee>
```

Figure 1 — Exécution de docker compose up -d et validation via docker ps : les 3 conteneurs Oracle XE sont Up sur les ports 1522, 1523 et 1524.

2. Configuration Réseau — 3 Machines

L'infrastructure repose sur une topologie multi-instances virtualisée via Docker, interconnectée à travers un réseau bridge isolé nommé oraclenet. Trois nœuds sont déployés :

- oracle-main (Serveur Central / Coordinateur) : Port hôte 1524 — point d'entrée unique garantissant la transparence de distribution.
- oracle-site1 (Fragment Nœud 1) : Port hôte 1522 — héberge les lignes avec Quantité $\geq$ 100 (Grossistes).
- oracle-site2 (Fragment Nœud 2) : Port hôte 1523 — héberge les lignes avec Quantité $<$ 100 (Détail).

Le réseau oraclenet (bridge) permet aux conteneurs de se résoudre mutuellement par nom d'hôte (oracle-main, oracle-site1, oracle-site2), ce qui est essentiel pour la configuration des Database Links.

3. Configuration du Site Principal

3.1 Connexion et création de l'utilisateur commun

L'utilisateur commun `c##bddvente` est créé sur l'instance Main (port 1524) avec les droits DBA nécessaires. Lors de la première tentative de connexion via Oracle SQL Developer, une erreur ORA-01017 (invalid username/password) a été levée car le script d'initialisation contenait la chaîne littérale 'VotreMotDePasse' à la place du mot de passe effectif (1111).

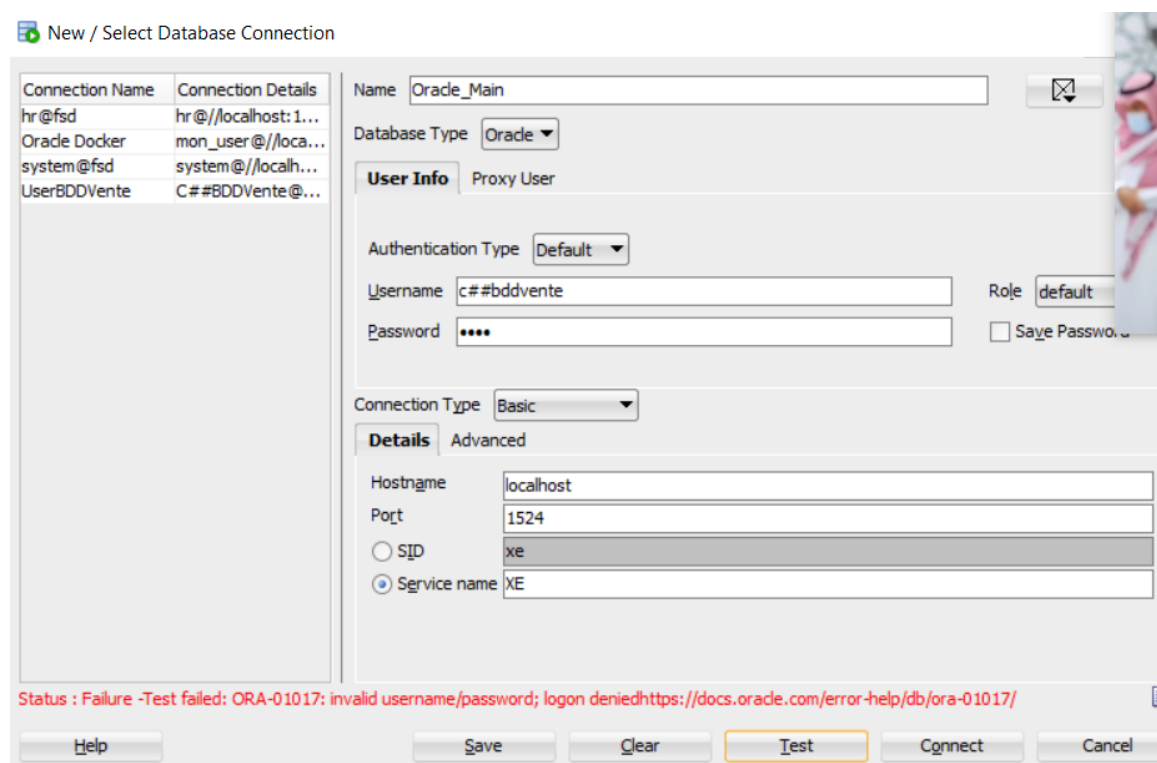


Figure 2 — Erreur ORA-01017 lors de la connexion Oracle_Main (port 1524) : mot de passe incorrect dans le script d'initialisation.

```
-- 1. Création de l'utilisateur sur l'instance principale
CREATE USER c##bddvente IDENTIFIED BY VotreMotDePasse;
```

Figure 3 — Script erroné contenant la variable littérale 'VotreMotDePasse' non substituée.

Résolution : reconnexion sous le rôle SYSDBA et réinitialisation du profil utilisateur avec `ALTER USER c##bddvente IDENTIFIED BY 1111`, puis attribution des droits `CONNECT`, `RESOURCE`, `DBA`.

4. Configuration des Sites Distants

4.1 Résolution de l'erreur ORA-01045 sur les shards

Bien que le schéma global soit partagé, l'utilisateur `c##bddvente` ne possédait pas initialement le droit de monter une session locale sur les deux sites distants, levant une erreur ORA-01045 (User lacks CREATE SESSION privilege).

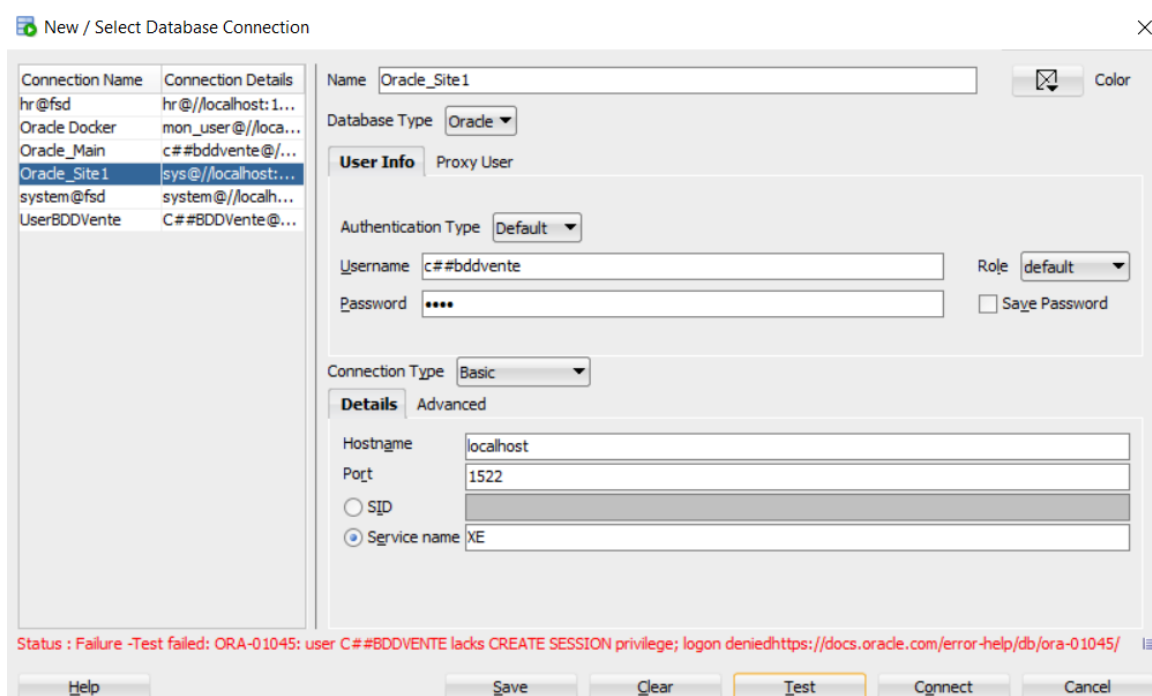


Figure 4 — Erreur ORA-01045 lors de la connexion Oracle_Site1 (port 1522) : absence du privilège CREATE SESSION.

Résolution : exécution de GRANT CONNECT, RESOURCE, DBA TO c##bddvente sur chaque shard (ports 1522 et 1523) sous le profil SYS AS SYSDBA. Suite à cette intervention, les connexions sont passées au statut 'Succès' sur les trois nœuds.

4.2 Structure des tables fragmentées

La table mère LIGNECOMMANDES possède 5 attributs. Les tables fragmentées lignecommandes1 et lignecommandes2 calquent cette signature exacte, avec une clé primaire et les contraintes référentielles vers commandes et produits locaux (ON DELETE CASCADE).

⚡	COLUMN_NAME	⚡	DATA_TYPE	⚡	NULLABLE	DATA_DEFAULT	⚡	COLUMN_ID	⚡	COMMENTS
1	IDLIGNECOMMANDE		NUMBER (38,0)		No	(null)		1		(null)
2	IDCOMMANDE		NUMBER (38,0)		Yes	(null)		2		(null)
3	IDPRODUIT		NUMBER (38,0)		Yes	(null)		3		(null)
4	QUANTITE		NUMBER (38,0)		Yes	(null)		4		(null)
5	REMISE		FLOAT		Yes	(null)		5		(null)

Figure 5 — Dictionnaire de données et métadonnées de la table mère LIGNECOMMANDES (5 attributs typés).

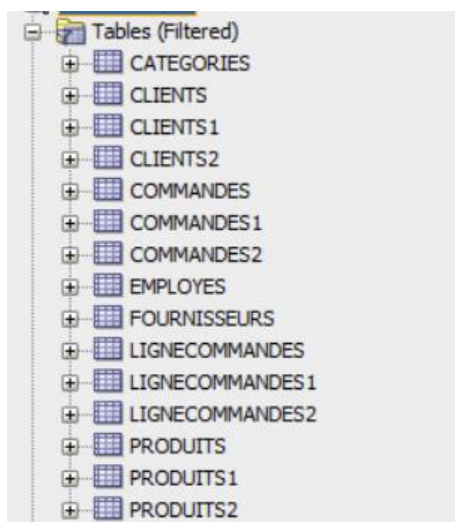


Figure 6 — Vue arborescente des tables du projet dans Oracle SQL Developer : tables fragmentées (LIGNECOMMANDES1/2, COMMANDES1/2, CLIENTS1/2, PRODUITS1/2) créées avec succès sur les deux sites.

4.3 Structure de la table PRODUITS (référence)

Name	Null?	Type
IDPRODUIT		NUMBER (38)
DESIGNATION		VARCHAR2 (100 CHAR)
IDFOUR		NUMBER (38)
IDCATEG		NUMBER (38)
PRIXUNITAIRE		FLOAT (126)
UNITESENSTOCK		NUMBER (38)
UNITESCOMMANDEES		NUMBER (38)
NIVEAUREAPPROVISIONNEMENT		NUMBER (38)
INDISPONIBLE		NUMBER (38)

Figure 7 — Structure de la table PRODUITS (colonnes et types) utilisée comme référence pour les fragments PRODUITS1 et PRODUITS2.

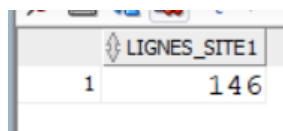
5. Tests de Connectivité

Après configuration des Database Links (voir section 7), les tests de connectivité valident la communication réseau entre les trois nœuds. Les fragments sont accessibles depuis l'instance Main via les liens l_site1 et l_site2.

Les comptages de lignes dans les deux fragments confirment la répartition correcte des données selon le critère de fragmentation :

	LIGNES_SITE2
1	320

Figure 8 — Résultat du comptage sur le fragment Site 2 : 320 lignes avec Quantité < 100 (LIGNES_SITE2).



LIGNES_SITE1	
1	146

Figure 9 — Résultat du comptage sur le fragment Site 1 : 146 lignes avec Quantité ≥ 100 (LIGNES_SITE1).

La répartition totale (146 + 320 = 466 lignes) correspond bien à l'ensemble des enregistrements de la table mère LIGNECOMMANDES, validant l'exhaustivité et la disjonction des fragments.

6. Création des Utilisateurs

L'utilisateur commun `c##bddvente` est créé avec des credentials identiques sur les trois instances Oracle (Main, Site1, Site2) pour assurer la transparence distribuée. La connexion initiale se fait en SYS AS SYSDBA sur chaque instance, puis les droits CONNECT, RESOURCE et DBA sont accordés.

Les figures 2 et 4 (sections 3 et 4) illustrent les connexions SQL Developer sur les trois nœuds. Une fois les utilisateurs correctement provisionnés, les connexions Oracle_Main (port 1524), Oracle_Site1 (port 1522) et Oracle_Site2 (port 1523) passent toutes en statut Succès.

7. Configuration des Database Links

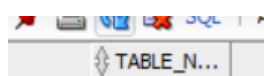
Depuis l'instance centrale oracle-main, les canaux de communication vers les shards sont configurés via des Database Links. L'usage d'un mot de passe purement numérique ('1111') a d'abord levé une erreur ORA-00988 (missing or invalid password). La solution consiste à encadrer le mot de passe de guillemets doubles pour forcer son analyse textuelle.

Deux liens sont créés : `l_site1` (vers oracle-site1:1521) et `l_site2` (vers oracle-site2:1521). Une vue logique globale `vue_global_lignecommandes` est ensuite créée sur le Main pour masquer la distribution aux utilisateurs finaux (UNION ALL des deux fragments distants).

8. Tables Fragmentées sur les Sites Distants

La fragmentation horizontale de la table LigneCommandes est basée sur le critère de quantité selon les requêtes R1 et R2 du scénario :

- R1 — Site 1 : $\sigma(\text{Quantite} \geq 100)$ — Grossistes / Entrepôt central — 146 lignes
- R2 — Site 2 : $\sigma(\text{Quantite} < 100)$ — Détail / Magasins de proximité — 320 lignes



TABLE_N...	
1	146

Figure 10 — Aperçu initial de la table `user_tables` (vide avant reconstruction des fragments).

Les tables fragmentées `lignecommandes1`, `commandes1`, `clients1` et `produits1` sont créées sur le Site 1, et leurs homologues sur le Site 2, avec les contraintes d'intégrité référentielle (clés primaires, clés étrangères) respectées sur chaque shard.

9. Architecture des Procédures Stockées

Des procédures d'encapsulation CRUD (Insert, Update, Delete) sont compilées sur chaque fragment. Elles garantissent qu'aucune application tierce ne peut altérer directement les lignes sans passer par la couche logique métier, et vérifient les contraintes d'intégrité référentielle avant toute opération.

`Procedure INSERT_COMMANDE1 compiled`

Figure 11 — Confirmation de compilation réussie : 'Procedure INSERT_COMMANDE1 compiled'.

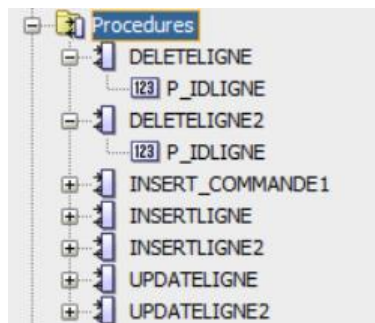


Figure 12 — Arborescence des procédures stockées dans SQL Developer : `DELETEDIGNE`, `DELETEDIGNE2`, `INSERT_COMMANDE1`, `INSERTLIGNE`, `INSERTLIGNE2`, `UPDATERIGNE`, `UPDATERIGNE2` compilées avec succès sur les deux sites.

Les procédures implémentées sur chaque site sont :

- `insertligne(p_idligne, p_idcommande, p_idproduit, p_quantite, p_remise)` : insertion avec vérification des contraintes d'intégrité référentielle (commande et produit existants).
- `deleteligne(p_idligne)` : suppression d'une ligne de commande par son identifiant.
- `updateligne(p_idligne, p_idproduit, p_quantite, p_remise)` : mise à jour des attributs `idproduit`, `quantite` et `remise`.

10. Architecture des Triggers

Trois triggers sont créés sur la table globale `LigneCommandes` du site Main pour intercepter les opérations DML et les router automatiquement vers le fragment approprié selon le critère de fragmentation (seuil `Quantité = 100`). Ils assurent la transparence totale de la distribution pour les applications clientes.

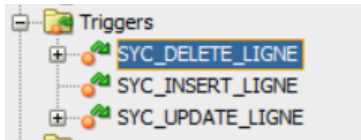


Figure 13 — Arborescence des triggers dans SQL Developer : SYC_DELETE_LIGNE, SYC_INSERT_LIGNE, SYC_UPDATE_LIGNE compilés et actifs.

- SYC_INSERT_LIGNE (AFTER INSERT) : Route l'insertion vers `lignecommandes1@l_site1` si Quantité ≥ 100 , sinon vers `lignecommandes2@l_site2`.
- SYC_DELETE_LIGNE (AFTER DELETE) : Propage la suppression sur les deux sites (par sécurité, afin de garantir la cohérence quel que soit le fragment contenant l'enregistrement).
- SYC_UPDATE_LIGNE (AFTER UPDATE) : Détecte si la quantité franchit le seuil 100 et effectue un transfert inter-fragment automatique : suppression du fragment d'origine et insertion dans le fragment cible.

11. Optimisation des Requêtes

11.1 Plan d'exécution initial — Full Table Scan

Avant optimisation, la requête de calcul du nombre de commandes par client en 2026 génère un TABLE ACCESS FULL sur la table Commandes. Le plan d'exécution révèle un coût de 19, avec un filtre appliqué sur `EXTRACT(YEAR FROM DATECOMMANDE) = 2026` sans support d'index.

PLAN_TABLE_OUTPUT							
1	Plan hash value: 4246182792						
2							
3	-----						
4	Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
5	-----						
6	0	SELECT STATEMENT		96	1152	19 (11)	00:00:01
7	1	HASH GROUP BY		96	1152	19 (11)	00:00:01
8	* 2	TABLE ACCESS FULL	COMMANDES	100	1200	18 (6)	00:00:01
9	-----						
10							
11	Predicate Information (identified by operation id):						
12	-----						
13							
14	2	filter(EXTRACT(YEAR FROM INTERNAL_FUNCTION("DATECOMMANDE"))=2026)					

Figure 14 — Plan d'exécution initial : TABLE ACCESS FULL sur COMMANDES, coût = 19, filtre sur `EXTRACT(YEAR FROM DATECOMMANDE) = 2026`.

11.2 Après optimisation — Index Range Scan

Un Function-Based Index (FBI) est créé sur `EXTRACT(YEAR FROM datecommande)`, permettant au moteur Oracle d'utiliser un INDEX RANGE SCAN au lieu du Full Table Scan. Les statistiques sont collectées manuellement via `DBMS_STATS` pour que le CBO (Cost-Based Optimizer) prenne en compte le nouvel index.

PLAN_TABLE_OUTPUT							
1 Plan hash value: 2617225364							
2							
3							
4	Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
5							
6	0	SELECT STATEMENT		96	1632	5 (20)	00:00:01
7	1	HASH GROUP BY		96	1632	5 (20)	00:00:01
8	2	TABLE ACCESS BY INDEX ROWID BATCHED	COMMANDES	100	1700	4 (0)	00:00:01
9	* 3	INDEX RANGE SCAN	IDX_CMD_ANNEE	40		1 (0)	00:00:01
10							
11							
12 Predicate Information (identified by operation id):							
13							
14							
15	3	- access(EXTRACT(YEAR FROM INTERNAL_FUNCTION("DATECOMMANDE"))=2026)					

Figure 15 — Plan d'exécution après création de `IDX_CMD_ANNEE` : `INDEX RANGE SCAN`, coût réduit de 19 à 5 (-74%). L'accès `TABLE ACCESS BY INDEX ROWID BATCHED` remplace le `Full Scan`.

12. Stratégie d'Indexation Multi-Niveaux

Pour optimiser les requêtes distribuées et locales, une stratégie d'indexation multi-niveaux est mise en place :

- Index Function-Based `IDX_CMD_ANNEE` sur `Commandes(EXTRACT(YEAR FROM datecommande))` : remplace le `Full Table Scan` par un `Index Range Scan` pour les filtres annuels (coût : 19 → 5).
- Index secondaires `idx_site1_idproduit` et `idx_site2_idproduit` sur les clés de jointure `idproduit` des tables fragmentées, accélérant les jointures distribuées entre produits et lignes de commandes.
- Index sur `idcommande` dans les tables fragmentées pour optimiser les jointures avec les tables commandes locales.

Les statistiques physiques sont régulièrement injectées dans le dictionnaire Oracle (CBO) via `DBMS_STATS.GATHER_TABLE_STATS` pour forcer le recalcul des chemins d'exécution optimaux.

13. Analyse Comparative des Performances

13.1 Chiffre d'Affaires par Produit — Catégorie 50

Pour calculer le chiffre d'affaires de la catégorie 50 depuis le Site 1, une stratégie hybride par vues (`View1` pour les données locales, `View2` pour les données distantes) est utilisée. Les résultats sont consolidés via `UNION ALL`.

	IDPRODUIT	DESIGNATION	CAT
1	182	eget	678146.6341269
2	280	Nulla	1081160.870244

Figure 16 — Résultats de la requête CA par produit (catégorie 50) : produit 182 (eget) = 678 146, produit 280 (Nulla) = 1 081 160.

13.2 Chiffre d'Affaires Global par Catégorie

La requête fédérée interroge simultanément et de façon transparente les structures produits1/2 et lignecommandes1/2 des deux fragments pour extraire le CA global par catégorie de produit.

IDCATEG	CHIFFRE_AFFAIRES_TOTAL
1	35 1789252.3804897
2	50 1569747.5605734

Figure 17 — Résultats du CA global par catégorie : Catégorie 35 = 1 789 252, Catégorie 50 = 1 569 747.

13.3 Plan d'exécution fédéré — Analyse

L'analyse du plan d'exécution sur la vue globale (WHERE idproduit = 182) depuis l'instance coordinatrice oracle-main met en évidence trois mécanismes clés :

Script Output x Query Result x

 All Rows Fetched: 21 in 0.105 seconds

PLAN_TABLE_OUTPUT

1 Plan hash value: 3708797963

2

3

4

5

6

7

8

9

10

11

12

13 Remote SQL Information (identified by operation id):

14

15

16

17

18

19

20

21

Figure 18 — Plan d'exécution distribué (oracle-main) : UNION-ALL + opérateurs REMOTE avec Predicate Pushdown vers L_SITE1 et L_SITE2, coût total = 4.

- Opérateur UNION-ALL (Id:2) : reconstruction logique transparente à partir des deux fragments physiques horizontaux.
- Opérateur REMOTE (Id:3 et Id:4) : aucun bloc de données n'est stocké localement sur le coordinateur — les appels sont délégués en temps réel via le réseau aux liaisons L_SITE1 et L_SITE2.
- Predicate Pushdown : le filtre WHERE IDPRODUIT=182 est poussé directement dans les requêtes SQL distantes (visible dans 'Remote SQL Information'). Seules les lignes qualifiées transitent sur le réseau oraclenet, réduisant le coût total à 4.

14. Monitoring et Maintenance des Performances

Le maintien des performances de l'architecture distribuée repose sur plusieurs axes de surveillance et de maintenance régulière :

14.1 Surveillance des index et statistiques

Les index créés (IDX_CMD_ANNEE, idx_site1_idproduit, idx_site2_idproduit) doivent être reconstruits périodiquement en cas de fragmentation physique. Les statistiques CBO sont collectées via DBMS_STATS.GATHER_SCHEMA_STATS('C##BDDVENTE') pour garantir des plans d'exécution optimaux au fil du temps.

14.2 Surveillance des Database Links

La disponibilité des liens l_site1 et l_site2 est critique. Tout incident réseau sur le bridge oraclenet entraîne une dégradation des requêtes distribuées. La vue ALL_DB_LINKS et des requêtes SELECT SYSDATE FROM DUAL@l_siteX permettent de vérifier la latence et la disponibilité.

14.3 Surveillance des sessions et performances

Les vues dynamiques v\$session et v\$lock permettent de détecter les sessions actives, les verrous bloquants et les requêtes longues. Le suivi des métriques de performance (temps de réponse, I/O, coût des plans) permet d'anticiper les dégradations avant qu'elles n'impactent les utilisateurs.

14.4 Maintenance des triggers et procédures

Les triggers SYNC_INSERT_LIGNE, SYNC_DELETE_LIGNE et SYNC_UPDATE_LIGNE ainsi que les procédures stockées doivent être revalidés après toute modification de structure des tables. La vue USER_OBJECTS permet de vérifier leur statut de compilation (VALID / INVALID).

Conclusion

Ce premier scénario a permis de valider concrètement l'ensemble des problématiques théoriques liées à la fragmentation horizontale des données dans un environnement Oracle XE distribué sous Docker.

- Infrastructure Docker Compose : trois conteneurs Oracle XE interconnectés via le réseau bridge oraclenet, avec validation opérationnelle confirmée par docker ps (Figure 1).
- Fragmentation horizontale : 466 lignes réparties en 146 (Site1, Quantité ≥ 100) et 320 (Site2, Quantité < 100), confirmant la disjonction et l'exhaustivité des fragments (Figures 8 et 9).
- Procédures stockées PL/SQL : les opérations CRUD (insertligne, deleteligne, updateligne) compilées et validées sur les deux sites (Figures 11 et 12).
- Triggers de synchronisation : SYNC_INSERT_LIGNE, SYNC_DELETE_LIGNE et SYNC_UPDATE_LIGNE actifs, assurant le routage automatique et le transfert inter-fragment au franchissement du seuil 100 (Figure 13).
- Optimisation des requêtes : l'index IDX_CMD_ANNEE réduit le coût d'exécution de 19 à 5 (-74%), transformant un Full Table Scan en Index Range Scan (Figures 14 et 15).
- Requêtes distribuées : le Predicate Pushdown Oracle pousse les filtres directement aux fragments distants, minimisant les transferts réseau (coût total = 4, Figure 18).

